

Examen réparti

Carlos Agon

Déroulement de L'examen

Tous les documents sont autorisés, et notamment ceux du cours.

L'utilisation de tout moyen de communication (téléphone, tablette, ordinateur personnel, etc.) est interdite.

L'utilisation d'une clé ou disque USB personnel, en lecture seule et étiqueté à votre nom, est autorisée; vous ne devez pas le prêter à un camarade.

Le barème pour chaque question n'est donné qu'à titre indicatif et pourra être adapté lors de la correction.

L'examen dure 2h.

Votre copie sera formée de fichiers textuels que vous laisserez aux endroits spécifiés dans votre espace de travail pour Eclipse. L'espace de travail pour Eclipse sera obligatoirement nommé `workspace` et devra être un sous-répertoire direct de votre répertoire personnel.

Le langage à étendre est ILP2. Le packaging Java correspondant sera nommé `com.paracamplus.ilp2.ilp2partiel`. Sera ramassé, à partir de votre `~/workspace` (situé sous ce nom directement dans votre HOME), le **seul** répertoire `~/workspace/ILP-UPMC/Java/src/com/paracamplus/ilp2/ilp2partiel/` et tous ses sous-répertoires.

L'examen sera corrigé à la main, il est donc absolument inutile de s'acharner sur un problème de compilation ou sur des méthodes à contenu informatif faible. Il est beaucoup plus important de rendre aisé, voire plaisant, le travail du correcteur et de lui indiquer, par tout moyen à votre convenance, de manière claire, compréhensible et terminologiquement précise, comment vous surmontez cette épreuve.

Mise en route

- Ouvrez Eclipse
- Choisissez le repertoire `workspace` comme `workspace`
- (facultatif, puisqu'on ne demande pas le code du compilateur) Compilez la bibliothèque d'exécution en faisant :

```
cd ~/workspace/ILP-UPMC/C/; make
```

- le plugin ANTLR devrait fonctionner sans réglages particuliers. Toutefois, si ce n'est pas le cas, vous pouvez toujours utiliser ANTLR en ligne de commande à l'aide de l'archive fournie dans les sources d'ILP, en faisant :

```
cd ~/workspace/ILP-UPMC/ANTLRGrammars
java -jar ~/workspace/ILP-UPMC/Java/jars/antlr-4.4-complete.jar \
-lib ~/workspace/ILP-UPMC/ANTLRGrammars \
```

```
-o ~/workspace/ILP-UPMC/target/generated-sources/antlr4/ \
LE_NOM_DE_VOTRE_GRAMMAIRE.g4
```

On rappelle que l'on peut aussi lancer le plug-in à la main depuis Eclipse avec "Run as..." si il ne se lance pas automatiquement lors de l'édition d'une grammaire.

Dépannage

— Si jamais eclipse se bloque en essayant de se connecter à svn, tapez dans un terminal :

```
killall eclipse; killall java
```

puis relancez eclipse.

Introduction

Dans cet examen, nous voulons étendre ILP2 en permettant la définition de *fonctions à appels comptés*. Une *fonction à appels comptés* est une fonction normale qui en plus compte le nombre de fois qu'elle est appelée. L'idée de cette extension est d'afficher (à la fin de l'exécution d'un programme) pour chaque *fonction à appels comptés* le nombre de fois qu'elle a été appelée. Bien sûr si le programme ne contient pas des *fonctionnes à appels comptés* on n'affichera rien.

Voici une exemple qui illustre cette extension :

```
function :comp deuxfois(x)
    ( 2 * x );

function puissancesix(x)
    (deuxfois(x) * deuxfois(x) * deuxfois(x));

let x = puissancesix in x(2);
```

Le résultat du programme précédent est 64, mais en plus il aura un effet de bord qui imprimera l'information suivante : "la fonction deuxfois a été appelée 3 fois". La différence entre la définition d'une fonction *classique* et une *fonction à appels comptés* se fait par l'ajout du mot clé **:comp** après le mot **function**.

1 Grammaire (2 points)

Ecrivez une grammaire dans le fichier :

com/paracamplus/ilp2/ilp2partiel/grammars/ILPMLgrammarPartiel.g4
permettant la définition des fonctions à appels comptés.

2 AST (3 points)

Proposez une extension de l'arbre syntaxique pour gérer l'introduction de *fonctions à appels comptés*. On rappelle que vous devez fournir les classes AST ainsi que les interfaces correspondantes dans com.paracamplus.ilp2.ilp2partiel.ast et com.paracamplus.ilp2.ilp2partiel.interfaces.

On n'oubliera pas d'inclure la classe ASTfactory correspondante et son interface IASTfactory ainsi que la nouvelle interface IASTvisitor.

3 Analyse syntaxique (2 points)

Ecrivez dans le package `com.paracamplus.ilp2.ilp2partiel.parser` les nouvelles classes `ILPMLparser` et `ILPMLlistener` permettant de construire les nouveaux nœuds ajoutés au langage.

4 Interprétation (5 points)

Implantez un nouvel intepeteur dans une classe

`com.paracamplus.ilp2.ilp2partiel.interpreter.Interpreter.java`.

5 Compilation (6 points)

5.1 Questions (2 points)

Répondez aux questions suivantes :

- listez les classes et interfaces que vous auriez à ajouter pour mettre en œuvre le compilateur, en précisant pour chacune les classes et interfaces qu'elles étendent ou implantent ;
- précisez quels aspects de la définition et l'invocation d'une *fonction à appels comptés* sont statiques, et lesquels sont dynamiques.

5.2 Schéma de compilation (4 points)

Proposez le ou les schémas de compilation pour l'invocation d'une *fonction à appels comptés* dans un fichier texte `com.paracamplus.ilp2.ilp2examen.compiler.Compiler.txt`

Si vous pensez qu'il faut faire des changements à la bibliothèque d'exécution, ajoutez les fichiers `.h` et `.c` nécessaires.

6 Tests (2 points)

Créez un répertoire `com/paracamplus/ilp2/ilp2partiel/SamplesPartiel/` dans lequel vous mettrez un ensemble de programmes pour tester votre implantation. Pour chaque programme expliquez l'erreur ou le comportement que vous testez.